Delay-Gated Telemetry-Aware Capacity Elasticity for SDN-Managed O-RAN Service Chains

Abstract

Delayed observations and queued resource commands can make elastic provisioning more expensive and less timely than fixed capacity. This paper studies that interaction in a reproducible, O-RAN-motivated three-stage service-chain model. A telemetry-aware controller projects reported queues using its applied-action ledger and pending commands, then enumerates a finite capacity-and-share grid. A deployment gate selects a separately calibrated fixed profile when configured actuation delay reaches the shorter class deadline. An evaluation frozen after development executes 800 policy runs across eight settings and twenty fresh seeds. At ten-slot actuation delay, the gated policy reduces mean incremental resource cost by 58.41–62.12% in stressed settings relative to calibrated fixed capacity. Class-1 mean miss fractions increase to 0.129–0.981%, compared with zero observed misses for fixed capacity in those fresh-seed settings. At long delays, the ungated predictor deteriorates, while the gated policy is pathwise identical to its fixed fallback by construction. The analysis establishes causality, conservation and finite-grid optimality for the decision surrogate, not stochastic deadline guarantees. Paired uncertainty, component ablations and negative results delineate when elasticity is useful. The results concern an abstract packet-service simulator, not a physical-layer or interoperable O-RAN implementation.

Keywords: Open RAN, software-defined networking, resource orchestration, delayed telemetry, delayed actuation, queue prediction, reproducible simulation.

1 Introduction

Open Radio Access Network (O-RAN) makes network control and orchestration an important part of radio-access-system design [**?**]. Software-defined networking (SDN) separates programmable control from forwarding functions [**?**], while virtualization changes how radio and computing resources are managed [**?**]. A practical consequence is that a resource decision need not take effect at the instant at which the relevant queue is measured. The controller may receive an old report, issue a command based on it, and then observe another report before that command becomes active. Treating this sequence as instantaneous obscures the difference between required and already-requested capacity.

Information age formalizes the distinction between a report's delivery time and the time represented by its contents [**?**]. Networked control likewise recognizes sensing and actuation delays as part of the closed-loop system [**?**]. Predictive control can explicitly account for future action effects, but its stability and optimality properties depend on the model and constraints used [**?**]. Consequently, adding a forecast to a queue heuristic does not establish a reliability guarantee. In particular, a predictor based on old service rates may continue requesting expansion that is already in transit. Conversely, an optimistic projection may release capacity before a burst has cleared.

The question considered here is deliberately narrower than full O-RAN resource optimization: *how do an action-aware queue projection, a bounded dispersion horizon and a delay-based deployment gate affect the cost–deadline trade-off under equal telemetry budgets?* The system contains two packet classes and three serial service stages. The stage interpretation is access, transport and compute, but each uses the same abstract packet-service unit. No radio propagation, physical resource block allocation or measured processor service curve is implied.

The contributions are as follows.

1. A causal controller uses asynchronously timestamped reports, an applied-action ledger and immutable pending commands to project workload at command activation. Its finite-grid decision separates surrogate feasibility from paid capacity cost.
2. A deployment policy gates that controller by the configured actuation delay. The fallback is selected independently by calibration, and its equivalence to the reference is established explicitly rather than presented as a novel safety guarantee.
3. A frozen comparison includes 800 executed runs, paired seeds, both classes, bounded-margin and ungated ablations, and long-delay failures. The evaluation distinguishes improvement over a legacy heuristic from improvement over fixed capacity.

4. An executable event contract, conservation checks, causality tests and independent packet replay connect the mathematical model to the reported outputs.

The intended contribution is a controlled computational study of delayed elasticity, not a claim that forecasting, O-RAN orchestration or policy fallback is new in isolation. The remainder positions the model against prior work, defines the controller and its limited formal properties, and evaluates its cost and deadline outcomes.

2 Related Work and Positioning

2.1 Orchestration and resource coupling

OrchestRAN selects intelligence algorithms and execution locations for specified network objectives [**?**]. OREO examines orchestration and sharing of xApp-based services [**?**]. These studies concern deployment of control intelligence, whereas the present decision changes the capacity and class reservations of packet-serving stages. Joint cloud, fronthaul and radio orchestration is already studied for cell-free massive multiple-input multiple-output systems [**?**]. Therefore, representing three resource domains does not itself establish novelty.

vrAIn learns coupled radio and computing allocations using an implemented virtualized radio access network (vRAN) [**?**]. Nuberu changes the distributed-unit processing pipeline to preserve operation under computing deficits [**?**]. Concordia predicts task execution requirements and schedules shared computing resources against stringent processing deadlines [**?**]. These systems motivate resource coupling but also expose a limitation of abstract tandem queues: physical radio and processor demand are not interchangeable packet counts.

Online learning for virtualized base-station allocation [**?**] and the delay-aware, multi-timescale MAREA orchestrator [**?**] address richer resource-allocation questions. MAREA's analytical delay treatment is distinct from the dispersion heuristic used below. Proactive artificial-intelligence and RAN workload orchestration [**?**] and learning-based functional-split selection [**?**] further show that anticipatory allocation and reconfiguration costs are established research directions. No published performance figure from these studies is imported as a baseline result in this paper.

2.2 Prediction, slicing and control interaction

Radio-intelligence-controller resource allocation [**?**], cloud-native multivariate probabilistic forecasting [**?**], and online-learning energy-efficient allocation [**?**] provide complementary approaches to resource decisions. The current predictor is neither a trained multivariate model nor an energy model. Its purpose is to isolate whether accounting for known actions changes a simple, fully inspectable controller.

COMIX addresses interactions among O-RAN xApps [**?**]. That problem differs from one controller's sequential commands but reinforces the need to distinguish requested actions from their combined effect. Service-enabled sub-slice isolation [**?**] and RadioSaber's channel-aware slicing with customizable schedulers [**?**] concern properties not captured by the two-class quota model. In particular, borrowing unused quota here does not prove slice isolation. EdgeRIC demonstrates the importance of control placement and timescale in an implemented system [**?**]. This motivates explicit delay sensitivity, without equating a synthetic slot with its real-time control cycle.

2.3 Experimental platforms and analytical boundaries

ColO-RAN [**?**], OpenRAN Gym [**?**], SCOPE [**?**], and FlexRIC [**?**] support experimental programmable-RAN work. ns-O-RAN integrates simulation with a RAN Intelligent Controller (RIC) [**?**], while Colosseum supports digital-twin experimentation [**?**]. The end-to-end 5G-LENA simulator [**?**] and its calibration studies [**?**] provide a route toward radio-aware evaluation. These platforms were not executed in the present experiments. Their relevance is methodological, not evidence of interoperability for this implementation.

Queue-based stochastic optimization [**?**], deterministic network calculus [**?**], and stochastic network calculus [**?**] distinguish long-run stability, worst-case delay and probabilistic guarantees. Scheduler analyses for interleaved weighted round-robin [**?**], deficit round-robin [**?**], and packet-level arrival constraints [**?**] further show that precise service assumptions matter. The controller below uses none of those service-curve guarantees. This distinction is important because a low surrogate score is not proof of an end-to-end packet deadline. Finally, verification of an implemented abstraction is separate from validation against its intended physical system [**?**].

Table 1: Closest-work distinctions. This is a conceptual comparison, not a reproduced benchmark.

Study	Principal decision or evidence	Distinction in the present study
OrchestRAN, OREO	Intelligence placement and xApp orchestration	Packet capacity, not xApp deployment
Demir et al.	Coupled physical cloud/fronthaul/radio resources	Three homogeneous service-unit stages
vrAIn, Concordia	Implemented compute/radio control and deadline scheduling	No calibrated CPU or physical radio model
Kalntis et al.	Online-learning vBS allocation	Deterministic finite-grid heuristic, no regret result
MAREA	Multi-timescale delay-aware radio orchestration	Deadline proxy, not an analytical violation bound
Probabilistic forecasting	Data-driven predictive resource information	Fixed-history, action-ledger projection
EdgeRIC	Low-latency control with a real RAN stack	Configured synthetic actuation-delay experiment
This study	Matched traces, pending commands, ablations and fallback	Conditional cost–delay evidence with explicit failure regimes

3 System Model and Problem Definition

3.1 State, service and event order

Let $i \in \{1,2,3\}$ index stages, $s \in \{0,1\}$ index classes and $t \in \{0,\dots,T-1\}$ index discrete slots. A packet has a class s, birth slot b and absolute deadline $b+D_s-1$. A packet completed in slot t has latency $t-b+1$ and is late if that value exceeds D_s. Let $Q_{is}(t)$ denote the queue at the beginning of slot t, before external arrivals, and let $A_s(t)$ denote newly generated packets. Service $S_{is}(t)$ uses total integer capacity $C_i(t) = c_i(t) + e_i(t)$, comprising base capacity c_i and active paid expansion e_i. The implemented dynamics are

$$Q_{1s}(t+1) = Q_{1s}(t) + A_s(t) - S_{1s}(t), \tag{1}$$

$$Q_{is}(t+1) = Q_{is}(t) - S_{is}(t) + S_{i-1,s}(t), \quad i = 2,3. \tag{2}$$

Stage 1 can serve new arrivals immediately. Stages 2 and 3 serve before their upstream neighbors, so a packet cannot traverse two stages in one slot. Each packet requires one service unit at every stage. Class queues are first-in, first-out (FIFO). There are no drops, retransmissions, alternate routes or finite-buffer admission decisions.

At each slot the engine applies due commands, delivers due reports, inserts external arrivals, requests a report and computes a command if the slot is a control epoch, serves stages in order $3,2,1$, and updates measurement histories. This ordering fixes otherwise consequential ambiguities at equal timestamps. Report snapshots are taken after external arrivals and before service. No new snapshot is visible to the controller before its configured delivery time.

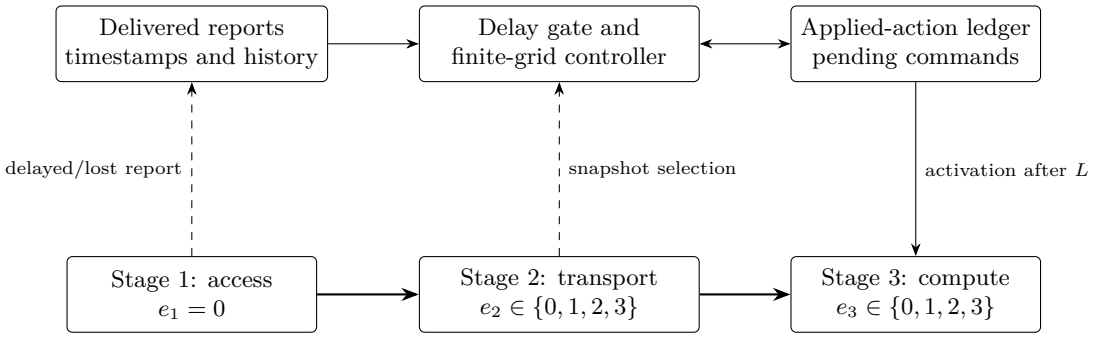


Figure 1: Implemented abstract service chain and information paths. All stages can report and receive class-share commands. Only stages 2 and 3 can expand capacity. The domain labels are an O-RAN motivation, not implemented protocol entities.

Table 2: Notation and units. All capacities and workloads are abstract.

Symbol	Definition
Q_{is}, A_s, S_{is}	Queue, external arrivals and service counts, in packets
c_i, e_i, C_i	Base, expansion and total capacity, in packets per slot
D_s, b, T	Class deadline, birth slot and run length, in slots
τ_i, Δ_i	Timestamp of latest delivered stage report and its age $t - \tau_i$
$\bar{a}_{is}, \bar{\mu}_{is}, v_{is}$	Historical mean ingress, mean service and variance of net ingress
h, W, R, L, p	Control interval, history window, report delay, activation delay and report loss probability
$\alpha, k, \mathcal{X}$	Class-0 share, share-grid index and candidate decision set
$\mathcal{L}, \mathcal{P}$	Applied-action ledger and pending-command list
$\widehat{q}_{is}, d_{is}, \rho_s$	Projected pre-service queue, demand proxy and normalized delay score
J, V_s	Full-run incremental cost and cohort deadline-miss fraction

3.2 Reservations and local borrowing

For weights $(\alpha, 1-\alpha)$, integer reserved service $g_{is}(C_i, \alpha)$ is obtained by flooring proportional quotas and assigning the remainder to the largest fractional part, with deterministic class-index tie breaking. Thus $\sum_s g_{is} = C_i$. The local scheduler first reserves

$$u_{is} = \min\{B_{is}, g_{is}\}, \qquad B_{1s} = Q_{1s} + A_s, \quad B_{is} = Q_{is} \ (i > 1). \tag{3}$$

Remaining units are borrowed by the eligible class whose next packet has the earliest absolute deadline, with class-index ties. This is deadline-based borrowing after reservations, not unrestricted earliest-deadline-first service. It ensures $\sum_s S_{is} = \min\{C_i, \sum_s B_{is}\}$.

3.3 Telemetry and immutable commands

Every $h = 5$ slots, one stage is selected for a report. Its payload is

$$r_i = (q_i, \bar{a}_i, \bar{\mu}_i, v_i, \bar{c}_i, \tau_i), \tag{4}$$

where q_i is the pre-service queue, $\bar{c}_i$ is the base capacity at the snapshot, and the rate statistics use the preceding $W = 20$ slots. Specifically, v_{is} is the population variance of stage ingress minus service over that window. Empty histories use fixed initialization values listed in the experiment setup. Each request is independently lost with probability p or delivered after R slots. A report older than the stored timestamp is discarded.

The common risk-based report selector uses

$$\widetilde{q}_{is}(t) = \left[q_{is} + \Delta_i(\bar{a}_{is} - \bar{\mu}_{is})\right]^+, \tag{5}$$

$$z_i(t) = \sum_s \frac{\sqrt{\Delta_i v_{is} + 1}}{D_s} \left(1 + \frac{\widetilde{q}_{is}(t)}{\max(1, \bar{c}_i)}\right) + 0.01\Delta_i. \tag{6}$$

The stage with greatest z_i is selected, breaking ties by stage index. The final age term is a heuristic priority term, not a bounded-starvation proof. Different methods may select different stages even though their request counts and request-index loss flags are identical.

A decision at time t creates an immutable command $(t+L, x, w)$ containing an expansion vector and stage-wise class weights. All due commands execute. There is no cancellation, acknowledgment loss, out-of-order actuation or supersession. Since L is fixed within each run, chronological decision order is preserved. Initial fixed capacity is preprovisioned, whereas elastic methods start with zero expansion. Fixed methods still update class shares through delayed commands.

3.4 Objective and measured reliability

The incremental resource objective is

$$J = \sum_{t=0}^{T-1} \sum_i e_i(t) + \kappa \sum_{t=1}^{T-1} \left\| e(t) - e(t-1) \right\|_1, \qquad \kappa = 0.25. \tag{7}$$

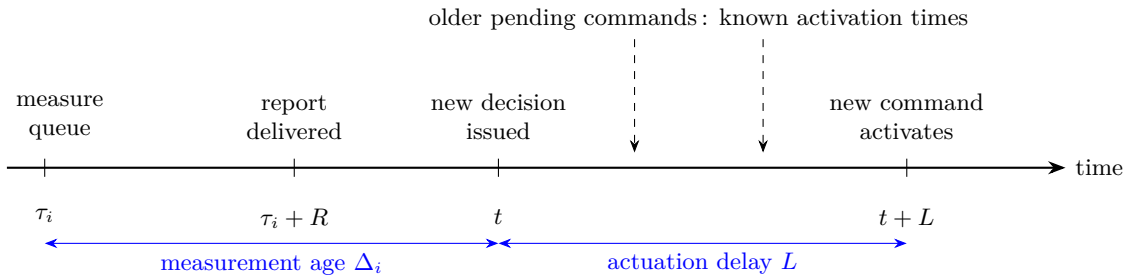


Figure 2: Projection spans report age and actuation delay. Commands already pending can change service before the new command activates. The new command itself is excluded when predicting the pre-service state at $t + L$.

It includes warm-up and tail capacity, charges actual expansion changes, and excludes base-resource costs. Initial provisioning incurs no switching charge. Units are abstract resource cost units, not money, energy or measured power.

For the measured birth cohort $\mathcal{C}_s$, let $N_s = |\mathcal{C}_s|$ and let ℓ_p be completion latency. The miss fraction is

$$V_s = \frac{\#\{p \in \mathcal{C}_s : \ell_p > D_s\} + \#\{p \in \mathcal{C}_s : p \text{ unfinished at run end}\}}{\max(1, N_s)}. \tag{8}$$

The tail extends beyond all measured deadlines, so unfinished cohort packets are known misses rather than right-censored successes. Background arrivals continue during that tail. Calibration uses a 1% mean-miss criterion for each class, but this finite-sample selection rule is not a chance constraint.

The ideal design problem minimizes expected J under classwise reliability requirements using only available information. The implemented method does not solve that stochastic control problem. It solves the per-decision surrogate defined next, and its actual reliability is measured separately.

4 Delay-Gated Action-Aware Controller

4.1 Projection from the action ledger

At decision time t, the ledger $\mathcal{L}(u)$ records the expansion and weights actually applied at slots $u \le t$. The list $\mathcal{P}(t)$ contains commands already issued but not yet activated. Their deterministic activation times define a known action trajectory up to $t + L - 1$. This uses no future exogenous arrivals or capacity realizations.

For each stage, initialize $\widehat{q}_i(\tau_i) = q_i$ and hold its latest reported base capacity and mean ingress constant. The fluid recurrence is

$$\widehat{q}_{is}(u+1) = [\widehat{q}_{is}(u) - F_s(\widehat{q}_i(u), w_i(u), \bar{c}_i + e_i(u))]^+ + \bar{a}_{is}, \quad u = \tau_i, \ldots, t + L - 1. \tag{9}$$

The fluid service operator F first applies integer reservations, limited by the real-valued projected workload, and assigns spare units in descending order of normalized residual workload $(\widehat{q}_{is} - u_{is})/D_s$. Ties use the class index. The projection does not contain packet ages, so this borrowing rule is only an approximation to the packet scheduler.

Each stage is projected independently using its last reported ingress. Projected upstream departures are not fed into the downstream forecast. This avoids pretending that asynchronously observed queues constitute a consistent instantaneous network state, but it also omits propagation of future upstream actions. The ledger is action-aware, not a perfect network predictor. The state at $t + L$ is pre-service and therefore excludes the effect of the new candidate command on that slot's service.

4.2 Finite-grid surrogate

The bounded-horizon demand proxy is

$$d_{is} = \widehat{q}_{is}(t + L) + h\bar{a}_{is} + \sqrt{hv_{is}} + 1. \tag{10}$$

The square-root term is a dispersion heuristic evaluated over one control interval. It is not a confidence bound on the longer queue-projection horizon. The additive one-packet term is a fixed conservative offset. Both are explicit design choices, not parameters inferred from O-RAN measurements.

Algorithm 1. Applied-action and pending-command projection

Input: reports r_i, time t, delay L, ledger $\mathcal{L}$, pending list $\mathcal{P}$
Construct known actions for $u = t, \dots, t + L - 1$ from $\mathcal{L}(t)$ and $\mathcal{P}$
for each stage $i = 1, 2, 3$ **do**
    Initialize $\widehat{q}_i \leftarrow q_i$ from the latest delivered report
    for $u = \tau_i, \dots, t + L - 1$ **do**
        Read applied action from $\mathcal{L}(u)$ if $u \le t$, else the known pending trajectory
        Compute fluid service F using reported base capacity and active expansion
        Update $\widehat{q}_i \leftarrow [\widehat{q}_i - F]^+ + \bar{a}_i$
    end for
end for
Return activation-time pre-service projections $\widehat{q}_i(t + L)$

The decision grid contains

$$\mathcal{X} = \{(x, \alpha) : x = (0, x_2, x_3),\ x_2, x_3 \in \{0, 1, 2, 3\},\ \alpha = k/20,\ k = 1, \dots, 19\}, \quad |\mathcal{X}| = 304. \tag{11}$$

The same share is applied at all stages. With reported base capacities, candidate integer reservations yield

$$\rho_s(x, \alpha) = \frac{1}{D_s} \sum_i \frac{d_{is}}{\max\{10^{-3}, g_{is}(\bar{c}_i + x_i, \alpha)\}}, \tag{12}$$

$$r(x, \alpha) = \max_s \rho_s(x, \alpha), \tag{13}$$

$$c(x) = \sum_i x_i + \kappa \left\| x - x^{\text{req}}_{\text{prev}} \right\|_1. \tag{14}$$

The small denominator floor defines a finite penalty for zero reservations, not guaranteed fractional service. The cost proxy compares with the previous *requested* expansion. It differs intentionally from the realized cost in (7), which uses actual activation changes.

Candidates are ranked lexicographically by

$$\sigma(x, \alpha) = \begin{cases} (0, c, r, \sum_s \rho_s, k), & r \le 1, \\ (1, r, c, \sum_s \rho_s, k), & r > 1. \end{cases} \tag{15}$$

The minimum is selected, retaining the earliest enumerated expansion on any remaining tie. Feasible candidates prioritize cost. When none is feasible, the least worst-class ratio is selected. No claim of feasibility is made for the realized packet system from this surrogate decision.

4.3 Deployment gate and comparators

Let x^{fix} be the separately calibrated fixed profile for the scenario, and define $G = \min_s D_s = 15$ slots. The proposed policy is

$$\pi_{\text{gate}}(L) = \begin{cases} \pi_{\text{pipe}}, & L < G, \\ \pi_{\text{fix}}, & L \ge G. \end{cases} \tag{16}$$

This choice is made before the run using its configured constant delay. The fallback starts preprovisioned and retains the fixed reference's class-share controller. It is not an online switch that instantly acquires capacity after latency rises. The threshold is an interpretable development choice, not a derived stability boundary.

The legacy comparator replaces the action projection with $[q_{is} + (\Delta_i + L)(\bar{a}_{is} - \bar{\mu}_{is})]^+$ and uses dispersion $\sqrt{(\Delta_i + L)v_{is}}$. The margin-only ablation retains that legacy drift projection but uses $\sqrt{hv_{is}}$. The ungated pipeline uses (9)–(15) at every delay. These are inspectable internal comparators, not reproductions of the published algorithms in Table 1.

5 Formal Properties and Limits

Proposition 1 (Causality). *For fixed configuration and initialization, every elastic decision at time t is a function only of reports delivered by t, commands already issued and their known activation times. It is independent of unobserved future traffic and base-capacity realizations.*

Algorithm 2. Delay-gated deployment and elastic decision

```
Input: configured delay L, deadline threshold G, calibrated profile x^fix
if L ≥ G then deploy fixed capacity and its reference share controller
    Return fixed-policy execution for the entire run
else initialize zero expansion, reports, action ledger and pending list
    at each slot apply due commands, deliver due reports and admit arrivals
        if t is a control epoch then
            Request one report using the common risk selector
            Project queues with Algorithm 1 and compute demand proxies
            Enumerate 304 candidates and minimize the ranking in (15)
            Enqueue the selected command for t + L without cancellation
        end if, serve stages 3, 2, 1 and update histories
end if
```

Proof. The report selector reads only stored payloads and their timestamps. The ledger contains previously applied commands. The pending list contains commands issued at earlier epochs. Equation (9) uses these actions and holds reported base capacity and ingress fixed. The demand and ranking equations are deterministic functions of that projection. Induction over decision epochs therefore excludes future exogenous trace entries. A future-suffix perturbation test additionally checks this property in the executable implementation. □

Proposition 2 (Conservation and service bounds)**.** *For every slot, all queues remain nonnegative, stage service does not exceed available capacity, and the total number of generated packets equals completed packets plus packets remaining in the chain. Completed latency is at least three slots.*

Proof. Reservations are limited by available queue contents. Borrowing only selects a class with a remaining packet and stops at capacity, proving nonnegativity and the service bound. Summing (1)–(2) cancels each internal transfer, leaving external arrivals minus final-stage completions. Induction from empty queues proves conservation. Reverse stage order prevents a packet from receiving two stages of service in one slot, giving the latency bound. □

Proposition 3 (Finite-grid decision optimality)**.** *The enumerator returns a minimum of (15) on $\mathcal{X}$. If any surrogate-feasible candidate exists, the result is surrogate-feasible and has minimum proxy cost among all such candidates.*

Proof. All sixteen expansion vectors and nineteen share values are visited. The running incumbent changes only for a strictly smaller lexicographic score. After the last candidate it is a minimum over the complete finite set. The leading score component ranks every feasible candidate ahead of every infeasible candidate, and cost is the first varying component within the feasible subset. This establishes the stated result, not global stochastic-control optimality. □

Proposition 4 (Pathwise fallback equivalence)**.** *For $L \geq G$, the gated policy and calibrated fixed reference have identical packet trajectories and metrics under the same exogenous trace and report-loss sequence.*

Proof. Both execute the same reference procedure with the same initial preprovisioned capacity, reports, weights, configuration and randomness. Their first-slot states are equal. If states and pending events are equal before a slot, the deterministic event contract produces equal updated states and events. Induction establishes identical service and completion trajectories and hence identical V_s and J. □

This result transfers the reference's behavior, including any misses. It does not improve or certify the reference's reliability.

5.1 Capacity obstruction and computational cost

A necessary condition for all packets in any cohort to meet their deadlines is that each stage has enough cumulative service opportunities within the union of their allowable residence intervals. In particular, if a cohort born in $[a, b]$ has N packets and maximum deadline $D_{\max}$, then

$$N \leq \sum_{t=a}^{b+D_{\max}-1} C_i(t) \qquad \text{for every stage } i. \tag{17}$$

Every such packet must consume one unit at every stage within that interval. Violation of (17) therefore precludes universal deadline satisfaction. The condition is loose and not sufficient because it ignores class deadlines, competing packets, stage precedence and service timing.

With two classes and three stages fixed, projection work scales with $\sum_i(\Delta_i + L)$. The grid requires 304 candidate scores, with reusable stage/share calculations. In the general m-stage extension with $E+1$ expansion levels at each expandable stage and K shares, exhaustive selection scales as $O(K(E+1)^{m-1}mS)$ for S classes. Thus the small present grid is not evidence of scalability to many independent resource domains. The reference implementation retains a full applied-action ledger. A production implementation could discard entries older than the oldest stored report, subject to its report delivery contract. No wall-clock real-time feasibility claim is made.

6 Experimental Method

6.1 Synthetic workload and calibration

All numerical parameters are synthetic design choices. A run contains 3340 slots, including 300 warm-up slots, a measured birth cohort spanning 3000 slots, and a 40-slot tail. A convenient 20 ms per slot mapping gives deadlines of 300 and 800 ms, not ultra-reliable low-latency communication targets. The fundamental reported delay unit remains the slot.

Table 3: Fixed experiment parameters. Poisson draws and capacity fluctuations are exogenous.

Parameter	Value
Classes, stages, deadlines	2, 3, $(D_0, D_1) = (15, 40)$ slots
Nominal Poisson arrival means	$(0.9, 2.1)$ packets per slot
Burst arrival means	$(1.8, 3.8)$ during $t \bmod 600 \in [180, 290)$
Base capacity	$5 + \xi$, with ξ sampled from $(-1, 0, 0, 1)$
Stressed-stage capacity	3 during $t \bmod 600 \in [170, 310)$
Control interval and history	$h = 5$ and $W = 20$ slots
Report delay/loss	$(R, p) = (3, 0.05)$ or $(9, 0.15)$
Actuation delays	$L \in \{1, 10, 25, 50, 100\}$ across specified cases
Expansion and share grid	$(0, x_2, x_3)$, $x_i \in \{0, 1, 2, 3\}$ and $\alpha = k/20$
Initial reported queues/capacity	$(0, 0)$ and 5 at each stage
Initial reported means/variance	Ingress $(0.9, 2.1)$, service $(2.5, 2.5)$, variance $(1, 1)$
Initial weights	Equal class weights at each stage
Switching weight and gate	$\kappa = 0.25$, $G = 15$ slots

The nominal case uses nominal arrivals and no capacity-stress window. Transport and compute cases introduce burst arrivals and stress stage 2 or 3, respectively. The delayed/lost-report case uses transport stress with $(R, p) = (9, 0.15)$. Five independent calibration seeds, 4000–4004, selected the cheapest of all sixteen fixed profiles with both class mean miss fractions at most 1%. The selected profiles were $(x_2, x_3) = (0, 0)$ for nominal, $(2, 0)$ for transport and delayed/lost reports, and $(0, 2)$ for compute. They were frozen before the current controller development and not retuned for the evaluation seeds or the longer activation delays. A fixed profile is therefore a strong calibrated comparator for its scenario, not an oracle that sees the test trace.

6.2 Development, locked evaluation and equal budgets

Development used seeds 8000–8004 at $L = 10$ and 100 for four variants, giving 40 runs. It revealed severe long-delay misses for the ungated action-aware projection and informed the deployment gate. These development results are retained in the reproducibility package and excluded from confirmatory summaries. The gate, fixed profiles, margin, objective and evaluation matrix were recorded before using fresh seeds 9000–9019.

The locked matrix contains the four original scenarios at $L = 10$ and ordinary transport stress at $L = 1, 25, 50, 100$. Each of these eight settings runs five methods on twenty seeds, producing $8 \times 5 \times 20 = 800$ executions. Gated and pipeline policies coincide below the gate, while gated and fixed coincide above it. These duplicated policy runs verify equivalence but do not increase the number of independent seeds.

Within a setting, every method receives the same arrivals and base capacities, verified by trace hashes. The same seeded request-index loss flags are used, and each method requests one report every five slots. Stage

selections can differ, and consequently identical delivered-report counts do not mean identical report contents. The paired design controls exogenous variation without giving any controller privileged queue information.

6.3 Uncertainty, testing and verification

Reported metrics average the twenty seed-level miss fractions and costs, rather than pooling all packets across seeds. Percentile bootstrap intervals use 10,000 seed resamples [**?**]. Differences use paired resampling of seed-level differences. The bootstrap unit is the independent seed, not an individual correlated packet. All-zero samples can yield a zero-width interval and do not establish zero true violation probability.

Exact two-sided sign tests exclude exact ties. A Holm adjustment [**?**] controls the declared family of 48 tests comprising eight settings, two comparators (legacy and fixed), and three endpoints (two miss fractions and cost). The sign test concerns the direction of paired differences, not their mean. Neither a nonsignificant test nor an interval spanning zero establishes equivalence or reliability noninferiority. Percentage-point differences are used for miss rates to avoid unstable relative changes near zero.

Verification comprises 43 unit-test methods, historical-regression checks, an independently implemented packet replay, an uncached enumeration reference, and reconciliation of new result records. The independent replay checks the packet engine against recorded actions, not the predictive model's physical validity. Dedicated tests exercise pending commands, activation-time boundaries, future-data exclusion, and gated-policy equivalence. All numerical tables and result plots below are generated directly from saved execution records, not manually entered performance targets.

7 Results

7.1 Primary settings at ten-slot actuation delay

Table 4 reports all five methods. In this regime, the deployment gate selects the pipeline method, so its identical rows are expected. The nominal case has zero fixed expansion cost, making percentage savings relative to that reference undefined. Any elastic activation there is an additional cost, not an efficiency gain.

Table 4: Primary results, twenty seeds per row. N10, T10, C10 and R10 denote nominal, transport, compute and delayed/lost reports at $L = 10$.

Setting	Method	Class 0 (%)	Class 1 (%)	Cost
N10	Legacy	0.0000	0.0000	173.45
N10	Pipeline	0.0000	0.0000	4.33
N10	Gated	0.0000	0.0000	4.33
N10	Margin-only	0.0000	0.0000	13.18
N10	Fixed	0.0000	0.0000	0.00
T10	Legacy	0.0000	0.0000	3829.76
T10	Pipeline	0.0000	0.1290	2773.80
T10	Gated	0.0000	0.1290	2773.80
T10	Margin-only	0.0208	0.0000	3061.10
T10	Fixed	0.0000	0.0000	6680.00
C10	Legacy	0.0000	0.0000	3861.43
C10	Pipeline	0.0016	0.1297	2777.95
C10	Gated	0.0016	0.1297	2777.95
C10	Margin-only	0.0224	0.0671	3062.97
C10	Fixed	0.0000	0.0000	6680.00
R10	Legacy	0.0473	0.0357	4087.15
R10	Pipeline	0.1076	0.9806	2530.55
R10	Gated	0.1076	0.9806	2530.55
R10	Margin-only	0.1507	0.7577	2950.38
R10	Fixed	0.0000	0.0000	6680.00

Across the three stressed settings, mean incremental cost savings of the gated policy relative to fixed

capacity are 58.41–62.12%. The corresponding class-1 mean miss fractions are 0.1290%, 0.1297% and 0.9806% for transport, compute and delayed/lost reports, respectively. The fixed reference has zero observed misses in these fresh-seed settings. In the delayed/lost-report case, the gated class-0 mean miss fraction is 0.1076%. Thus cost reduction is obtained by accepting additional deadline violations, and the data do not support a reliability-equivalence claim. The conclusion is therefore a cost–deadline trade-off within the evaluated workload family, not a uniform ordering of policies. The scatter plot in Fig. **??** makes that trade-off visible without combining cost and misses into a post hoc scalar performance score.

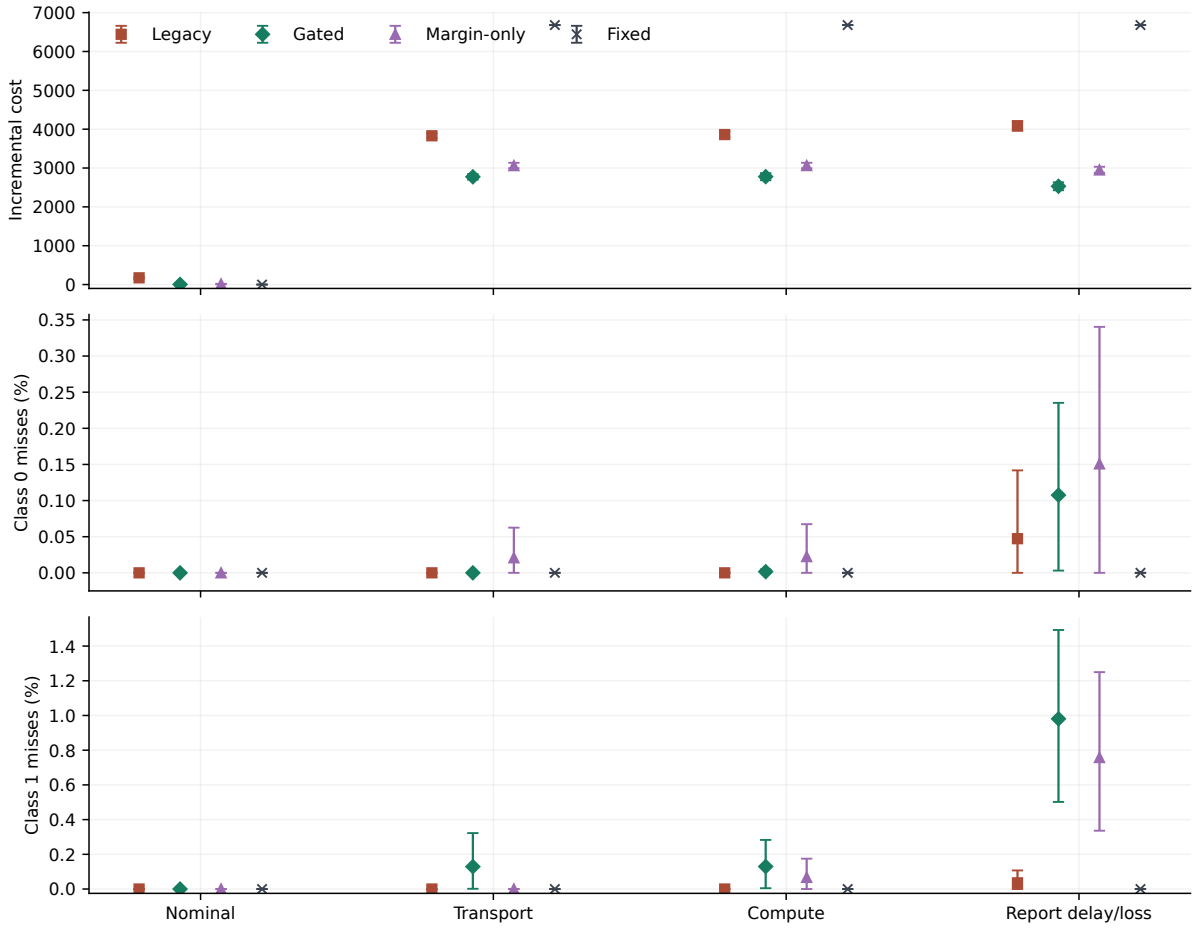


Figure 3: Primary comparison at $L = 10$. Error bars are seed-bootstrap 95% intervals. Gated equals pipeline in these settings. Cost is incremental and does not include the base service infrastructure.

7.2 Ablation and prediction limits

The margin-only method changes the dispersion horizon without adding an action ledger. Comparing it with legacy isolates that change. Comparing pipeline with margin-only then assesses the action-aware projection conditional on the same bounded margin. This sequential ablation does not identify every possible interaction. A ledger-only variant was examined during development but is not part of the five-method locked matrix.

At long delay the full ungated predictor remains vulnerable. A known action trajectory does not reveal future bursts or future base-capacity changes, and independent stage projection does not preserve dynamic coupling between stage departures and downstream ingress. The bounded margin also stops growing with the projection horizon. Consequently, projection awareness alone cannot be interpreted as robustness to arbitrary sensing and actuation delay.

7.3 Activation-delay sensitivity and gated fallback

Figure **??** includes transport stress at all five evaluated actuation delays. At $L \geq 15$, the gated implementation executes the fixed reference from the start. Its overlay with the reference is an identity of policy definition.

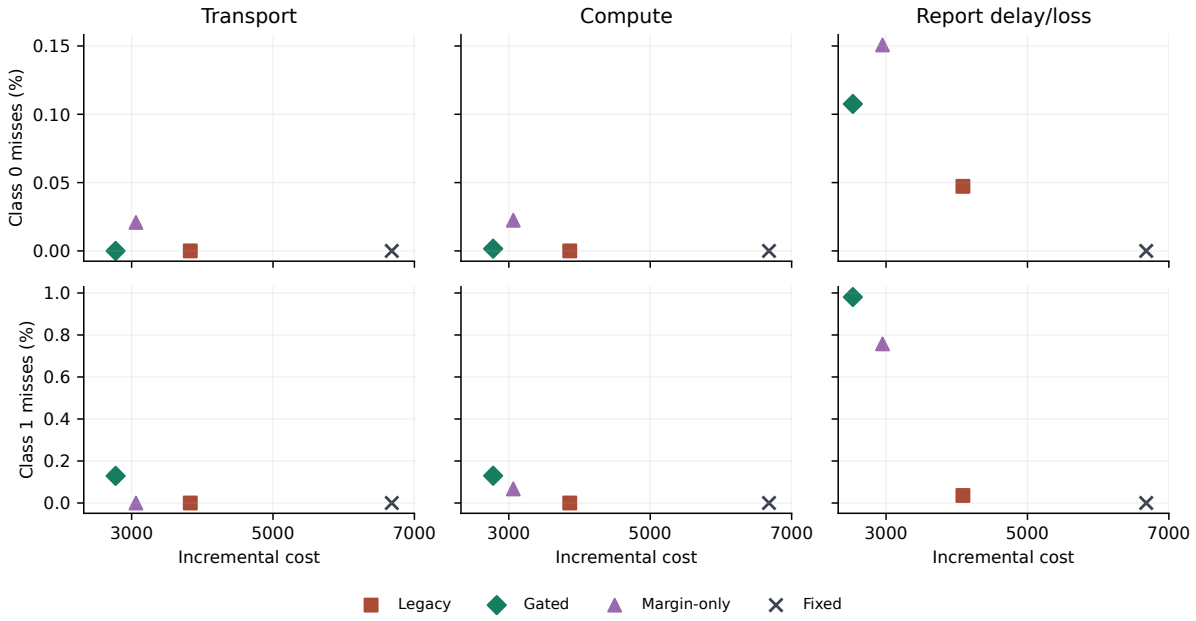


Figure 4: Cost–deadline outcomes in the three stressed primary settings. Each marker is a twenty-seed mean, not a Pareto-optimality certificate. The two rows retain both classes rather than hiding the class with the worse trade-off.

It shows what behavior the deployment gate preserves, not an independently learned long-delay improvement over fixed provisioning.

Table 5: Long-delay transport results. Class columns are mean deadline-miss percentages, not reliability guarantees.

Setting	Method	Class 0 (%)	Class 1 (%)	Cost
T25	Legacy	0.0000	0.0000	4600.26
T25	Pipeline	0.4361	1.7755	2867.18
T25	Gated	0.0000	0.0000	6680.00
T25	Margin-only	0.2186	0.2350	3347.57
T25	Fixed	0.0000	0.0000	6680.00
T50	Legacy	0.2492	1.9390	6482.31
T50	Pipeline	0.9216	15.4091	3299.35
T50	Gated	0.0000	0.0000	6680.00
T50	Margin-only	2.2980	10.3856	3723.46
T50	Fixed	0.0000	0.0000	6680.00
T100	Legacy	0.0000	6.2103	11010.51
T100	Pipeline	1.2337	26.1533	4283.27
T100	Gated	0.0000	0.0000	6680.00
T100	Margin-only	3.6957	25.4715	4503.29
T100	Fixed	0.0000	0.0000	6680.00

At $L = 100$, the ungated pipeline has class-0 and class-1 mean miss fractions of 1.234% and 26.153%, with cost 4283.27. Legacy class-1 misses reach 6.210% at cost 11010.51. Gated and fixed both cost 6680.00 and have zero observed cohort misses in these twenty runs. The gate avoids deploying that ungated heuristic in this configured regime, at the price of retaining fixed expansion cost. The experiment does not locate the optimal threshold. There are no locked performance runs at $L = 14$ or 15, and no delay that changes during a run. Thus neither continuity of performance at the gate nor safe online switching follows from these results.

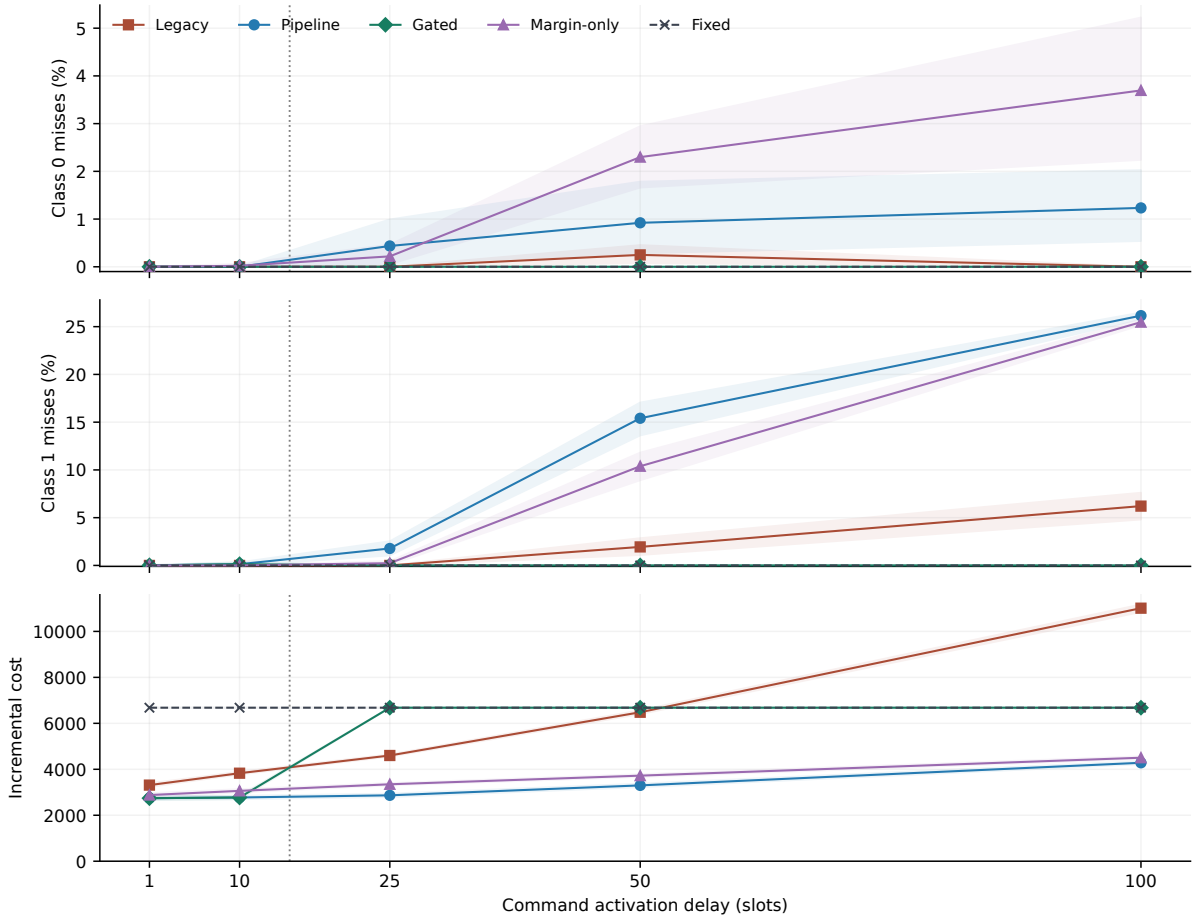


Figure 5: Transport-stress activation-delay sensitivity. Shading denotes seed-bootstrap 95% intervals. The dashed vertical line marks the deployment threshold, not a proven stability boundary. Fixed and gated curves overlap above the threshold.

7.4 Paired comparisons

Figure ?? reports paired cost differences for the primary settings. Negative values favor the gated method. Appendix ?? includes all 48 comparisons, their confidence intervals and Holm-adjusted sign-test values, including ties and non-significant outcomes. This prevents selective emphasis on favorable cost results while omitting classwise reliability.

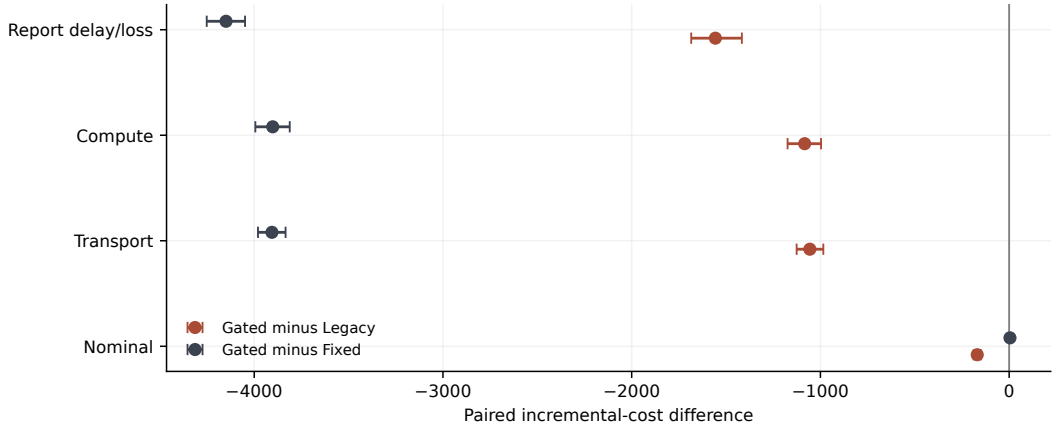


Figure 6: Paired mean incremental-cost difference with seed-bootstrap 95% intervals at $L = 10$. These are differences of matched runs. They are not differences between independently resampled method means.

8 Discussion and Threats to Validity

The experiments support treating the pending command pipeline as part of an elastic controller's state. However, useful short-delay behavior does not generalize to arbitrarily long delays. The gate makes this limitation explicit at deployment and preserves a previously selected reference when the chosen delay criterion is not met. Its simplicity is useful for auditability, but it does not constitute a new general stability theorem or an optimal switching law.

Model validity. The service units do not map to physical resource blocks, transport bit rates or CPU cycles. Access, transport and compute labels identify serial bottlenecks, not an implemented O-RAN functional split. The 20 ms mapping is illustrative. No SDN southbound protocol, RIC E2 interface, radio channel, mobility, retransmission or interoperability test was executed. The numerical savings cannot be restated as energy or financial savings.

Workload and selection validity. Bursts are periodic and Poisson rates, capacity windows, class deadlines and switching prices are fixed design choices. Twenty fresh seeds assess stochastic variation within this family, not representativeness of operator traffic. Fixed profiles were calibrated on five seeds, leaving selection uncertainty. The development gate may be suboptimal outside its tested settings. Additional traffic families, expansion prices, history lengths and gate thresholds require separately frozen experiments.

Algorithmic validity. The fluid predictor lacks packet ages, forecasts stage ingress independently, and holds reported base capacity constant. The surrogate uses reserved quotas while actual local borrowing is work-conserving. Its feasibility can therefore disagree with actual deadline behavior. The cost proxy penalizes requested changes rather than optimizing the full future activation sequence. Commands have deterministic delay and reliable execution, which excludes cancellation, reordering and resource-instantiation failures.

Comparative validity. The baseline set isolates the proposed mechanism and contains a calibrated fixed reference, but it does not reproduce MAREA, vrAIn, published online learners or an offline stochastic-control oracle. Accordingly, the results establish no state-of-the-art superiority. The absence of a physical validation experiment and of directly reproduced closest-work baselines materially limits the strength of any journal-submission claim.

Statistical validity. Bootstrap intervals are finite-sample estimates. Packet correlation is handled by seed-level resampling, but workload-family uncertainty is not. The sign test and mean confidence interval answer different questions. Reliability equivalence is proved only for identical fallback implementations, not inferred from nonsignificant tests for distinct methods.

An O-RAN validation extension should map each abstract resource to a measured service curve, record telemetry and activation timestamps, reproduce the same loss-and-delay perturbations in a RIC-connected simulator or testbed, and verify actual resource-accounting units. Only then should the deployment gate be recalibrated against measured latency and service objectives. A quantum component is not required by the present problem and is not added without a concrete algorithmic role and a defensible classical comparison.

9 Conclusion

This paper examined delayed telemetry and queued actuation in a controlled three-stage resource-elasticity model. The action-ledger predictor and bounded dispersion horizon change the short-delay cost–deadline trade-off, while the ungated method fails to remain reliable under long actuation delays. A fixed deployment gate preserves a calibrated reference in that regime by exact policy equivalence. The resulting evidence supports conditional use of elasticity, not universal superiority or guaranteed deadlines. The executable model, paired fresh-seed evaluation, complete negative results and explicit formal limits provide a reproducible basis for subsequent physical O-RAN validation.

Data and Code Availability

The accompanying reproducibility package contains the simulator, controller, development records, calibration and historical results, frozen evaluation protocol, all 800 run records, statistical procedures, manuscript source, vector plots and verification receipts. The main result file is `paper_results.json`. Running `build_paper.py` regenerates the manuscript's numeric tables and figures from that file. No external dataset or unexecuted testbed output is represented as experimental evidence.

A Complete Paired Statistical Results

Case abbreviations are N (nominal), T (transport), C (compute) and R (delayed/lost reports). A suffix is the actuation delay in slots. Differences are gated minus comparator. Class endpoints are percentage points, cost is abstract units. $n_{\neq 0}$ counts non-tied pairs. All-zero differences give $p = 1$ by convention.

Table 6: All paired comparisons. Intervals are unadjusted 95% seed-bootstrap intervals.

	Case	Reference	Endpoint	Mean difference	95% interval	$n_{\neq 0}$	p_{Holm}
N10	Legacy	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
N10	Legacy	Class 1		0.0000	[0.0000, 0.0000]	0	1.000
N10	Legacy	Cost		-169.1250	[-187.8128, -150.1247]	20	0.00009
N10	Fixed	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
N10	Fixed	Class 1		0.0000	[0.0000, 0.0000]	0	1.000
N10	Fixed	Cost		4.3250	[2.2000, 7.2000]	11	0.034
T10	Legacy	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T10	Legacy	Class 1		0.1290	[0.0014, 0.3224]	4	1.000
T10	Legacy	Cost		-1055.9625	[-1125.9503, -984.6375]	20	0.00009
T10	Fixed	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T10	Fixed	Class 1		0.1290	[0.0014, 0.3224]	4	1.000
T10	Fixed	Cost		-3906.2000	[-3980.1000, -3833.8991]	20	0.00009
C10	Legacy	Class 0		0.0016	[0.0000, 0.0048]	1	1.000
C10	Legacy	Class 1		0.1297	[0.0048, 0.2830]	4	1.000
C10	Legacy	Cost		-1083.4750	[-1174.0028, -996.5681]	20	0.00009
C10	Fixed	Class 0		0.0016	[0.0000, 0.0048]	1	1.000
C10	Fixed	Class 1		0.1297	[0.0048, 0.2830]	4	1.000
C10	Fixed	Cost		-3902.0500	[-3994.1131, -3811.8991]	20	0.00009
R10	Legacy	Class 0		0.0603	[-0.0158, 0.1703]	4	1.000
R10	Legacy	Class 1		0.9449	[0.4904, 1.4325]	11	0.034
R10	Legacy	Cost		-1556.6000	[-1684.5650, -1416.2113]	20	0.00009
R10	Fixed	Class 0		0.1076	[0.0031, 0.2352]	4	1.000
R10	Fixed	Class 1		0.9806	[0.5019, 1.4927]	11	0.034
R10	Fixed	Cost		-4149.4500	[-4251.7125, -4048.5734]	20	0.00009
T1	Legacy	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T1	Legacy	Class 1		0.0118	[0.0000, 0.0340]	2	1.000
T1	Legacy	Cost		-568.2750	[-618.9756, -517.5491]	20	0.00009
T1	Fixed	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T1	Fixed	Class 1		0.0118	[0.0000, 0.0340]	2	1.000
T1	Fixed	Cost		-3936.9000	[-4009.2756, -3852.4363]	20	0.00009
T25	Legacy	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T25	Legacy	Class 1		0.0000	[0.0000, 0.0000]	0	1.000
T25	Legacy	Cost		2079.7375	[2032.8241, 2127.4253]	20	0.00009
T25	Fixed	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T25	Fixed	Class 1		0.0000	[0.0000, 0.0000]	0	1.000
T25	Fixed	Cost		0.0000	[0.0000, 0.0000]	0	1.000
T50	Legacy	Class 0		-0.2492	[-0.4542, -0.0794]	7	0.484
T50	Legacy	Class 1		-1.9390	[-2.8498, -1.0957]	14	0.004
T50	Legacy	Cost		197.6875	[115.8834, 278.6628]	20	0.082
T50	Fixed	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T50	Fixed	Class 1		0.0000	[0.0000, 0.0000]	0	1.000
T50	Fixed	Cost		0.0000	[0.0000, 0.0000]	0	1.000
T100	Legacy	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T100	Legacy	Class 1		-6.2103	[-7.6081, -4.8087]	20	0.00009
T100	Legacy	Cost		-4330.5125	[-4490.0294, -4163.5106]	20	0.00009
T100	Fixed	Class 0		0.0000	[0.0000, 0.0000]	0	1.000
T100	Fixed	Class 1		0.0000	[0.0000, 0.0000]	0	1.000
T100	Fixed	Cost		0.0000	[0.0000, 0.0000]	0	1.000

B Reproduction Contract

Use Python 3 with NumPy and Matplotlib for artifact generation, and a LaTeX installation with the packages declared in the source. Simulation and most verification routines use the Python standard library. Run the unit tests and `verify_paper.py` before regenerating the manuscript. The evaluation script can resume complete case checkpoints. For a clean independent replay, run it in a new directory containing the source files but no checkpoint. The frozen protocol digest is embedded in the result file. Package checksums identify the delivered versions rather than claiming that the study was externally preregistered. The archive retains earlier technical reports as dated development history. This manuscript is the current paper artifact.

References

[1] M. Polese et al., "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," author version, arXiv:2202.01032, 2022. 10.48550/arXiv.2202.01032.

[2] D. Kreutz et al., "Software-Defined Networking: A Comprehensive Survey," Proceedings of the IEEE, vol. 103, no. 1, 2015. 10.1109/JPROC.2014.2371999.

[3] A. Garcia-Saavedra and X. Costa-Perez, "O-RAN: Disrupting the Virtualized RAN Ecosystem," IEEE Communications Standards Magazine, 2021. 10.1109/MCOMSTD.101.2000014.

[4] R. D. Yates et al., "Age of Information: An Introduction and Survey," author version, arXiv:2007.08564, 2020. 10.48550/arXiv.2007.08564.

[5] J. P. Hespanha, P. Naghshtabrizi, and Y. Xu, "A Survey of Recent Results in Networked Control Systems," Proceedings of the IEEE, vol. 95, no. 1, pp. 138–162, 2007. 10.1109/JPROC.2006.887288.

[6] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," Automatica, vol. 36, no. 6, pp. 789–814, 2000. 10.1016/S0005-1098(99)00214-9.

[7] S. D'Oro, L. Bonati, M. Polese, and T. Melodia, "OrchestRAN: Network Automation through Orchestrated Intelligence in the Open RAN," IEEE INFOCOM, 2022, author version arXiv:2201.05632. 10.48550/arXiv.2201.05632.

[8] F. Mungari, C. Puligheddu, A. Garcia-Saavedra, and C. F. Chiasserini, "OREO: O-RAN intElligence Orchestration of xApp-based network services," author version, arXiv:2405.18198, 2024. 10.48550/arXiv.2405.18198.

[9] O. T. Demir, M. Masoudi, E. Bjornson, and C. Cavdar, "Cell-Free Massive MIMO in O-RAN: Energy-Aware Joint Orchestration of Cloud, Fronthaul, and Radio Resources," IEEE Journal on Selected Areas in Communications, vol. 42, no. 2, pp. 356–372, 2024. 10.1109/JSAC.2023.3336187.

[10] J. A. Ayala-Romero et al., "vrAIn: A Deep Learning Approach Tailoring Computing and Radio Resources in Virtualized RANs," ACM MobiCom, 2019. 10.1145/3300061.3345431.

[11] G. Garcia-Aviles et al., "Nuberu: Reliable RAN virtualization in shared platforms," ACM MobiCom, 2021. 10.1145/3447993.3483266.

[12] X. Foukas and B. Radunovic, "Concordia: Teaching the 5G vRAN to Share Compute," ACM SIGCOMM, 2021. Author/publisher source.

[13] M. Kalntis, G. Iosifidis, and F. A. Kuipers, "Adaptive Resource Allocation for Virtualized Base Stations in O-RAN With Online Learning," IEEE Transactions on Communications, 2024. 10.1109/TCOMM.2024.3461569.

[14] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, and X. Costa-Perez, "MAREA: A Delay-Aware Multi-time-Scale Radio Resource Orchestrator for 6G O-RAN," IEEE Transactions on Communications, 2025. 10.1109/TCOMM.2025.3552296.

[15] S. D. A. Shah, M. Hafeez, A. Salama, and S. A. R. Zaidi, "Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks," author version, arXiv:2507.09124, 2025. 10.48550/arXiv.2507.09124.

[16] F. W. Murti, S. Ali, G. Iosifidis, and M. Latva-aho, "Learning-Based Orchestration for Dynamic Functional Split and Resource Allocation in vRANs," EuCNC/6G Summit, 2022. 10.1109/EuCNC/6GSummit54941.2022.9815815.

[17] Q. Wang et al., "Resource Allocation Based on Radio Intelligence Controller for Open RAN Toward 6G," IEEE Access, vol. 11, pp. 97909–97919, 2023. 10.1109/ACCESS.2023.3311888.

[18] V. Kasuluru et al., "Cloud-Native Multivariate Probabilistic Forecasting for Next-Generation Resource Allocation in O-RAN," IEEE Access, vol. 14, pp. 2904–2923, 2026. 10.1109/ACCESS.2025.3648065.

[19] A. Tuerxun and A. Nakao, "SORA: Energy-Efficient Resource Allocation in Open Radio Access Network With Online Learning," IEEE Access, vol. 13, pp. 113686–113701, 2025. 10.1109/ACCESS.2025.3584345.

[20] A. E. Giannopoulos et al., "COMIX: Generalized Conflict Management in O-RAN xApps—Architecture, Workflow, and a Power Control Case," IEEE Access, vol. 13, pp. 116684–116700, 2025. 10.1109/ACCESS.2025.3585774.

[21] M. Iqbal and A. Nakao, "5ERASI: 5G Service-Enabled Resource Allocation and Sub-Slice Isolation for Open RAN," IEEE Access, 2026. 10.1109/ACCESS.2026.3693783.

[22] Y. Chen, R. Yao, H. Hassanieh, and R. Mittal, "Channel-Aware 5G RAN Slicing with Customizable Schedulers," USENIX NSDI, pp. 1767–1782, 2023. Author/publisher source.

[23] W.-H. Ko et al., "EdgeRIC: Empowering Realtime Intelligent Optimization and Control in NextG Networks," author version, arXiv:2304.11199, 2023. 10.48550/arXiv.2304.11199.

[24] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "ColO-RAN: Developing Machine Learning-based xApps for Open RAN Closed-loop Control on Programmable Experimental Platforms," author version, arXiv:2112.09559, 2022 revision. 10.48550/arXiv.2112.09559.

[25] L. Bonati, M. Polese, S. D'Oro, S. Basagni, and T. Melodia, "OpenRAN Gym: AI/ML Development, Data Collection, and Testing for O-RAN on PAWR Platforms," Computer Networks, 2023. 10.1016/j.comnet.2022.109502.

[26] L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "SCOPE: An Open and Softwarized Prototyping Platform for NextG Systems," ACM MobiSys, 2021. 10.1145/3458864.3466863.

[27] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: an SDK for next-generation SD-RANs," ACM CoNEXT, pp. 411–425, 2021. 10.1145/3485983.3494870.

[28] A. Lacava et al., "ns-O-RAN: Simulating O-RAN 5G Systems in ns-3," ACM Workshop on ns-3, 2023. 10.1145/3592149.3592161.

[29] M. Polese et al., "Colosseum: The Open RAN Digital Twin," author version, arXiv:2404.17317, 2024. 10.48550/arXiv.2404.17317.

[30] N. Patriciello, S. Lagen, B. Bojovic, and L. Giupponi, "An E2E Simulator for 5G NR Networks," Simulation Modelling Practice and Theory, vol. 96, article 101933, 2019. Author/publisher source.

[31] K. Koutlia, B. Bojovic, Z. Ali, and S. Lagen, "Calibration of the 5G-LENA System Level Simulator in 3GPP reference scenarios," Simulation Modelling Practice and Theory, vol. 119, article 102580, 2022. Author/publisher source.

[32] M. J. Neely, "Stochastic Network Optimization with Application to Communication and Queueing Systems," Morgan & Claypool, 2010, Springer electronic edition. 10.1007/978-3-031-79995-2.

[33] J.-Y. Le Boudec and P. Thiran, "Network Calculus: A Theory of Deterministic Queuing Systems for the Internet," Springer, 2001. 10.1007/3-540-45318-0.

[34] Y. Jiang and Y. Liu, "Stochastic Network Calculus," Springer, 2008. 10.1007/978-1-84800-127-5.

[35] S. M. Tabatabaee, J.-Y. Le Boudec, and M. Boyer, "Interleaved Weighted Round-Robin: A Network Calculus Analysis," author version, arXiv:2003.08372, 2020. 10.48550/arXiv.2003.08372.

[36] S. M. Tabatabaee and J.-Y. Le Boudec, "Deficit Round-Robin: A Second Network Calculus Analysis," author version, arXiv:2106.01034, 2021. 10.48550/arXiv.2106.01034.

[37] E. Mohammadpour, E. Stai, and J.-Y. Le Boudec, "Improved Network Calculus Delay Bounds in Time-Sensitive Networks," author version, arXiv:2204.10906, 2022. 10.48550/arXiv.2204.10906.

[38] R. G. Sargent, "Verification and validation of simulation models," Journal of Simulation, vol. 7, pp. 12–24, 2013. 10.1057/jos.2012.20.

[39] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979. 10.1214/aos/1176344552.

[40] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," Scandinavian Journal of Statistics, vol. 6, pp. 65–70, 1979. Author/publisher source.